Lab 5 Report: Localization

Team 4

Christian Cassamajor-Paul
Alicia Ramirez
Leo Sun
Raymond Wang

RSS

April 12, 2026

1 Introduction

A big part of autonomous navigation is localization, or, knowing the vehicle's location and heading before making meaningful planning or control decisions. In this lab, we designed a Monte Carlo Localization (MCL) system for the RACECAR platform that estimates the car's 2D pose in a known map using noisy odometry and LiDAR measurements. This state-estimation layer acts as the connection between perception and planning/control. It turns raw motion and sensing data into a usable pose estimate, and supports the next stage of autonomous navigation where path planning and path following rely on good localization. The purpose of the lab was to implement, tune, and evaluate a real-time particle filter in both simulation and on the physical car. Challenges in this lab include overcoming odometry drift, processing noisy LiDAR scans, and localizing in spaces with few distinct features or with features that look alike. Our solution is to maintain many pose hypotheses, propagate them forward with a motion model, compare predicted and observed LiDAR ranges with a sensor model, and repeatedly reweight and resample the hypotheses so they converge to the most likely pose. This design lets the system stay robust to uncertainty while producing a fast, consistent localization estimate that downstream autonomous modules can trust.

2 Technical Approach

2.1 Particle Filter Initialization - Christian

We initialize a particle cloud of N particles from a user-provided pose (via the RVIZ 2D Pose Estimate button). The input pose is converted from $(x, y, \text{quaternion})$ to (x, y, θ) where θ is the yaw. Each particle is then generated by adding Gaussian noise $[\epsilon_x, \epsilon_y, \epsilon_\theta] \sim \mathcal{N}(0, 0.1)$ to this pose, producing a cloud of hypotheses centered around the initial estimate. The standard deviation was chosen via qualitative analysis of particle spread in simulation.

2.2 Motion Model - Christian

The motion model is responsible for updating the various pose hypotheses based on the latest odometry data. By injecting noise into the motion model, we aim to cover a range of possible updates that represent all the potential poses that the racecar may have.

On every odometry update after the cloud has been initialized, the motion model will apply the motion from the odometry to each particle, respectively, to that particle's frame, and then add further noise to each component of the pose to represent the noisiness and subsequent uncertainty of real-life odometry data. To be more specific, the following process occurs:

1. Odometry data received as a vector: $[\Delta x, \Delta y, \Delta\theta]$.
2. Use the pose info from timestep t-1 of each particle to transform and apply the odometry in the particle's frame for the current timestep t with the following equations:

$$x_t = x_{t-1} + \Delta x \cos(\theta_{t-1}) - \Delta y \sin(\theta_{t-1}) \tag{1}$$

$$y_t = y_{t-1} + \Delta x \sin(\theta_{t-1}) + \Delta y \cos(\theta_{t-1}) \tag{2}$$

$$\theta_t = \theta_{t-1} + \Delta\theta \tag{3}$$

3. Sample from a Gaussian distribution and then add noise for each particle in a similar manner to the initialization to ensure particles spread and cover a range of potential poses.

The Nx3 array of particles that results from the above process is returned to the particle filter and used in later operations as explained in Section 2.4.

2.3 Sensor Model - Ray and Alicia

The sensor model computes the likelihood of the observed LiDAR scan given a particle's pose. In the MCL pipeline, this likelihood is used as the particle's weight during resampling, so particles whose predicted measurements better match the observed scan are more likely to be retained.

2.3.1 Beam Model

The beam model decomposes $p(z \mid d)$—the probability of measuring range z when the true (ray-cast) range is d—into a mixture of four distributions:

$$p(z \mid d) \;=\; \alpha_{\text{hit}}\, p_{\text{hit}}(z \mid d) + \alpha_{\text{short}}\, p_{\text{short}}(z \mid d) + \alpha_{\text{max}}\, p_{\text{max}}(z) + \alpha_{\text{rand}}\, p_{\text{rand}}(z) \tag{4}$$

- **Hit** (p_{hit}): A Gaussian centered at the true range d:

$$p_{\text{hit}}(z \mid d) = \eta_{\text{hit}}\, \exp\!\left(-\frac{(z-d)^2}{2\sigma_{\text{hit}}^2}\right), \quad z \in [0, z_{\text{max}}] \tag{5}$$

- **Short** (p_{short}): Accounting for unexpected obstacles that return shorter-than-expected ranges:

$$p_{\text{short}}(z \mid d) = \frac{2}{d}\left(1 - \frac{z}{d}\right), \quad z \le d \tag{6}$$

- **Max** (p_{max}): A point mass at z_{max}, representing complete sensor failures or highly reflective surfaces.
- **Rand** (p_{rand}): A uniform distribution $1/z_{\text{max}}$ over the full range, capturing random returns.

The mixing weights were tuned empirically to $\alpha_{\text{hit}} = 0.69$, $\alpha_{\text{short}} = 0.12$, $\alpha_{\text{max}} = 0.07$, $\alpha_{\text{rand}} = 0.12$, with $\sigma_{\text{hit}} = 0.5$ (in discretized pixel units). We have comparatively greater weight on α_{short} to trust the motion model more due to many non-mapped obstacles in Stata.

2.3.2 Precomputed Lookup Table

For efficiency we precomputed all values of Equation 4 into a table T, where entry $T[z, d]$ stores the column-normalized probability $p(z \mid d)$. During each update, measured and ray-cast ranges are converted from meters to integers for direct table indexing.

2.3.3 Particle Weight Computation Using Lookup Table

For particle $\mathbf{x}_i$, RangeLibc performs ray casting through the occupancy grid to produce expected ranges $\mathbf{d}_i = (d_i^1, \ldots, d_i^B)$. Assuming conditional independence across beams, the log-likelihood is:

$$\log p(\mathbf{z}_t \mid \mathbf{x}_i, m) \;=\; \sum_{b=1}^{B} \log T\big[z_t^b,\; d_i^b\big] \tag{7}$$

To prevent the probabilities from collapsing on a few particles we compute a geometric mean:

$$w_i \;=\; \exp\!\left(\frac{1}{B}\sum_{b=1}^{B}\log\, T\big[z_t^b,\; d_i^b\big]\right) \tag{8}$$

This retains sensor information while still discriminating among poses.

2.4 Particle Filter Update Loop - Leo

Our node separates the Bayes filter into an odometry-driven prediction step and a LiDAR-driven correction step. On each odometry callback, we compute a control increment

$$\mathbf{u}_t = \begin{bmatrix} v_x \Delta t \\ v_y \Delta t \\ \omega \Delta t \end{bmatrix}, \tag{9}$$

where v_x, v_y are body-frame linear velocities, ω is the yaw rate, and Δt is the elapsed time. Each particle is then propagated through the motion model:

$$\mathbf{x}_t^{[i]} \sim p(\mathbf{x}_t \mid \mathbf{u}_t, \mathbf{x}_{t-1}^{[i]}). \tag{10}$$

On each LiDAR callback, we downsample the scan to $B = 100$ beams and compute unnormalized weights using the geometric-mean formulation from Equation 8:

$$\tilde{w}_t^{[i]} = \exp\!\left(\frac{1}{B}\sum_{b=1}^{B}\log\, T\big[z_t^b,\; d_i^b\big]\right), \tag{11}$$

which are then normalized:

$$w_t^{[i]} = \frac{\tilde{w}_t^{[i]}}{\sum_{j=1}^{N}\tilde{w}_t^{[j]}}. \tag{12}$$

To balance convergence speed against particle impoverishment, we resample stochastically on roughly half of the sensor updates. When resampling is not triggered, new likelihoods are multiplied into existing weights to accumulate evidence. When resampling is triggered, ancestor indices are drawn from the normalized weights:

$$a_i \sim \text{Categorical}\Big(w_t^{[1]}, \ldots, w_t^{[N]}\Big), \tag{13}$$

and each new particle is copied from its ancestor with a small Gaussian perturbation to preserve diversity:

$$\mathbf{x}_t^{[i]} \leftarrow \mathbf{x}_t^{[a_i]} + \boldsymbol{\epsilon}_i, \qquad \boldsymbol{\epsilon}_i \sim \mathcal{N}\big(\mathbf{0}, \text{diag}(\sigma_x^2, \sigma_y^2, \sigma_\theta^2)\big). \tag{14}$$

After each update, the final pose estimate is the weighted mean over all particles:

$$\hat{x}_t = \sum_{i=1}^{N} w_t^{[i]} x_t^{[i]}, \qquad \hat{y}_t = \sum_{i=1}^{N} w_t^{[i]} y_t^{[i]}, \tag{15}$$

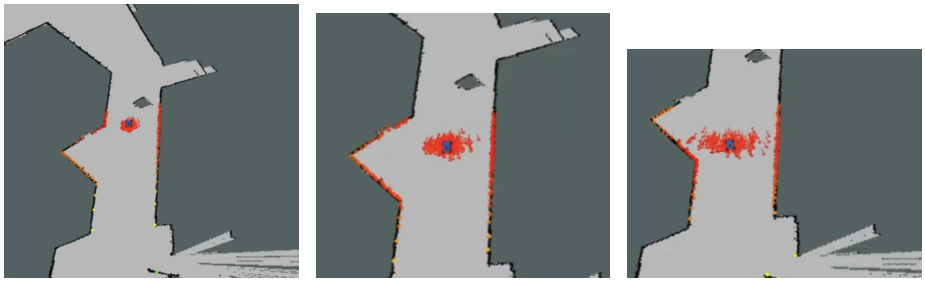
$$\hat{\theta}_t = \text{atan2}\left(\sum_{i=1}^{N} w_t^{[i]} \sin\theta_t^{[i]},\ \sum_{i=1}^{N} w_t^{[i]} \cos\theta_t^{[i]}\right). \tag{16}$$

where the circular mean via atan2 handles angle wrapping.

3 Experimental Evaluation

3.1 Motion Model Evaluation - Christian

The motion model was evaluated, individually, without the sensor model, within the RVIZ simulation environment. We performed numerous tests involving initializing the car and then teleoperating it around the map to empirically determine the quality of the motion model. This approach was also used to tune the standard deviation value of the Gaussian noise that gets sampled by the motion model. By looking at how much the particles spread within a given amount of time, it becomes very clear which direction the standard deviation value should be pushed in so that the particles remain reasonable estimates of potential poses.



(a) Motion model at time t seconds. Each red arrow represents a pose in the particle cloud of pose estimates.

(b) Motion model at t+1 seconds. Arrows have begun to spread, but still remain reasonably close together.

(c) Motion model at t+2 seconds. Arrows spread even further as the uncertainty of the true position grows.

Figure 1: Motion model test run. Noise with standard deviation = 0.15, mean = 0.

By leveraging ROS bags, we are able to repeat each test as many times as we like for greater precision and more valid results.

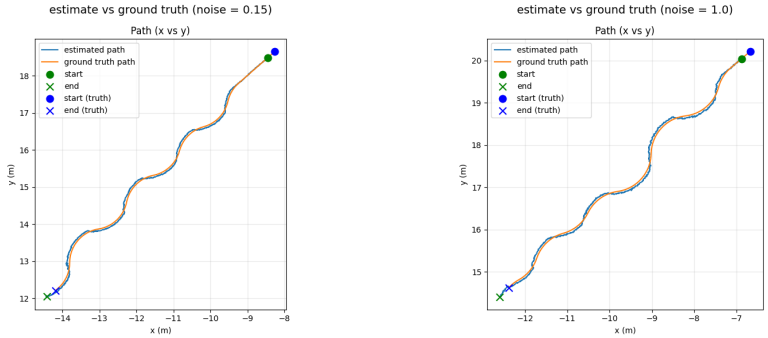
3.2 Particle Filter Evaluation (Simulation) - Christian

We evaluated the particle filter in simulation by recording ROS bags of four separate tele-operated tests and then comparing the ground truth result to the estimated location that the particle filter gave. We were able to efficiently do

this as the simulation environment provides us with the actual position of the car in simulation via the map to baselink transform that gets published to \tf. We calculated the average x error and y error by subtracting the estimated value from the true value at every point and then taking the average of each. This results in the average errors seen in table 1. Notably, as noise levels increase, the average error increases as well. The poorer performance of the higher noise can also be seen in the increasing jaggedness of the estimated path within the plots in Figures 2 and 3.

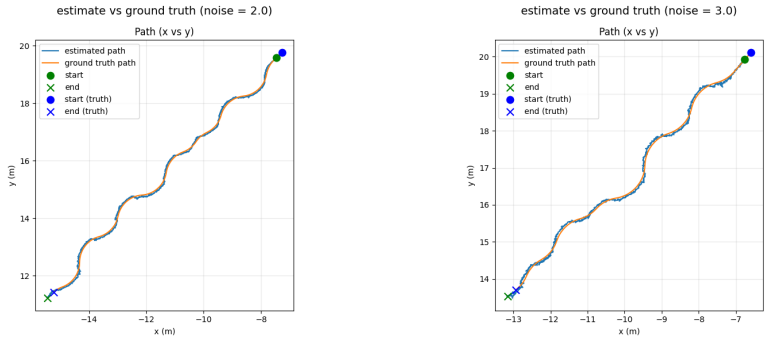
Noise level (std dev)	**avg x_err (m)**	**avg y_err (m)**
0.15	0.197	0.187
1.0	0.194	0.187
2.0	0.195	0.202
3.0	0.202	0.206

Table 1: Average position error for different noise levels



(a) (b)

Figure 2: Plots of estimated localization over a trajectory compared to the ground truth.

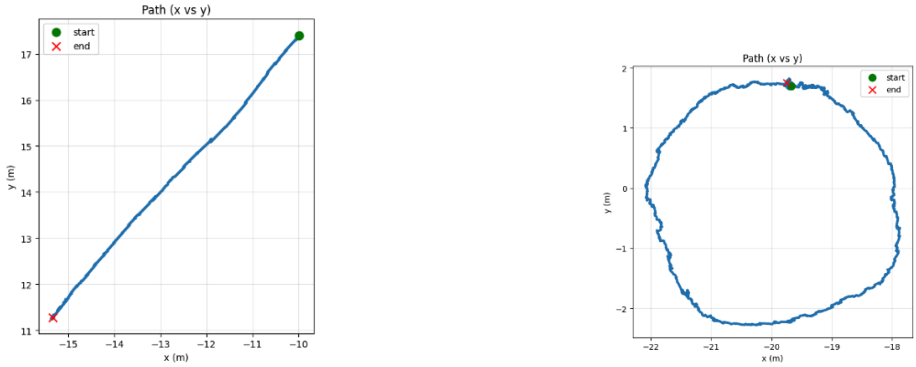


(a) (b)

Figure 3: Plots of estimated localization over a trajectory compared to the ground truth.

3.3 Particle Filter Evaluation (Reality) - Leo

To evaluate localization accuracy on the physical car, we designed two repeatable tests: a straight-line drive and a closed-loop circular drive. In both cases, we compared the particle filter's pose estimate against ground-truth positions measured with a tape measure and translated into map coordinates using the Foxglove measurement tool. Each test was repeated across five trials. In the straight-line test, the car was driven forward from a known start to a marked endpoint, and we computed the Euclidean error between the estimated and measured positions. In the circular test, the car was driven around the orange circle in the Stata basement and returned to its starting position; any discrepancy between the start and end estimates reflects accumulated drift over the loop.



(a) Straight-line test (b) Circular test

Figure 4: Real-world localization test examples.

Across both tests, the particle filter achieved an RMSE of approximately 9 cm.

Because the car was driven manually via teleop, it did not stop exactly on the marked endpoint in every trial, so the true localization accuracy is likely slightly better than reported. One notable difficulty arose in a section of the Stata basement with significant clutter and stacked boxes that were not present in the occupancy grid map. In this area, the LiDAR returns differed substantially from the ray-cast expectations, causing particle weights to degrade and the estimate to drift until the car moved into a region with better map agreement.

4 Conclusion - Leo

Our final Monte Carlo Localization system combines a noisy odometry-based motion model, a LiDAR beam-based sensor model, and a particle filter that maintains pose hypotheses over time. We achieved real-time performance through vectorized operations, LiDAR downsampling, sensor model precomputation, and careful resampling tuning.

Experiments showed the particle filter could recover from uncertainty, converge to the correct pose, and provide stable estimates for downstream modules. However, performance depends heavily on parameter tuning, map quality, and environmental structure, especially on the physical car, where dynamic obstacles and sensing imperfections are more noticeable.

Future improvements could include adaptive resampling, better cross-environment parameter tuning, and stronger robustness to dynamic scenes. With localization now in place, the system is ready for path planning and autonomous navigation.

5 Lessons Learned

Christian Cassamajor-Paul

I learned that good documentation is necessary for keeping essential knowledge accessible to the team.

Alicia Ramirez

I learned that finishing the code early so that we have half a week for testing is very helpful, and that making a testing plan beforehand speeds everything up.

Leo Sun

I learned that rosbags can be used not only to collect data, but also to increase working efficiency.

Raymond Wang

I learned that modularized testing makes debugging and interpreting results much faster.